Web5: Local-First Applications with Chain-Anchored Trust

Zach Kelling*
Zoo Labs Foundation Lux Industries Hanzo AI
research@zoo.ngo

April 2025

Abstract

We present Web5, an application architecture that combines local-first data ownership with blockchain-anchored trust. Unlike Web3 approaches that attempt to store application state on-chain, Web5 keeps mutable data in per-user encrypted SQLite shards, synchronizes across devices via Conflict-free Replicated Data Types (CRDTs), wraps encryption keys with post-quantum ML-KEM, and commits only Merkle roots and policy state to the chain. The result is a system with the user experience of local applications, the portability of decentralized identity, the security of cryptographic permissions, and the auditability of blockchain settlement. We describe a concrete implementation using the Lux Network's VM architecture (IdentityVM, KeyVM, ThresholdVM) as the trust plane, Hanzo Base as the local runtime, and ZAP as the transport protocol. The system achieves sub-10µs sync latency, per-user data isolation via deterministic key derivation (HMAC-SHA256 DEK/KEK hierarchy), and offline-first operation with eventual consistency. We provide formal definitions, a reference SDK with 18 passing tests, and deployment configurations for the Zoo Network as a reference application.

1 Introduction

The evolution of web application architectures can be characterized by where state lives and who controls trust:

- **Web1**: Read-only. State on the server. Trust by domain authority.
- **Web2**: Read-write. State on someone else's servers. Trust by platform identity (OAuth, SAML).
- **Web3**: Shared state on blockchains. Trust by consensus. But: slow, public, expensive for mutable app data.
- **Web5**: User-owned local state. Portable identity. Cryptographic permissions. Chain-anchored interoperability.

The key insight is that blockchains are excellent as *trust planes*—for identity, keys, policy, settlement, and audit—but poor as *data planes* for the mutable, private, high-frequency state that applications actually need.

Web5 separates these concerns:

- **Data plane**: Encrypted SQLite shards, CRDT ops, blob storage, device cache, app workers.
- **Trust plane**: DIDs, keys, policies, threshold approvals, Merkle anchors, payments, proofs.

This paper describes the architecture, implementation, and deployment of this model.

*zach@zoo.ngo

2 Architecture

2.1 Personal Runtime

Every user, organization, and application gets a local-first runtime:

1. Encrypted SQLite shard (one file per user/org)
2. CRDT synchronization log (conflict-free multi-device merge)
3. Local cache and indexes
4. Media and blob storage references
5. Device key material (DEK cached locally, zeroed on revocation)

SQLite is an ideal unit of tenancy because it supports single-writer semantics (no contention between users), portable file format, and embedded operation without external dependencies [1].

2.2 Key Hierarchy

We employ a three-level key hierarchy:

$$\text{OrgKEK} = \text{HMAC-SHA256}(\text{MasterKEK}, \texttt{"vault:org:"} \| \text{orgID}) \tag{1}$$

$$\text{UserDEK} = \text{HMAC-SHA256}(\text{OrgKEK}, \texttt{"vault:user:"} \| \text{userID}) \tag{2}$$

The Master KEK resides in a Hardware Security Module (AWS KMS, GCP Cloud KMS, Azure Key Vault) or is threshold-reconstructed via the K-Chain's ML-KEM key governance. It never touches disk.

Each user's DEK encrypts their SQLite shard using AES-256-GCM with random 12-byte nonces per entry. This provides:

- **User isolation**: Compromising one DEK reveals only that user's data.
- **Org isolation**: Different org KEKs produce different DEKs for the same user ID.
- **Rotation**: Org KEK rotation requires only re-deriving DEKs and re-encrypting shard headers, not re-encrypting all row data.
- **Revocation**: Zeroing a DEK from device memory immediately prevents further reads.

For post-quantum security, the Master KEK can be wrapped using ML-KEM-768 (NIST FIPS 203) [2] for key establishment between HSM, K-Chain validators, and user devices.

2.3 CRDT Synchronization

Multi-device access is enabled by Conflict-free Replicated Data Types (CRDTs). Our implementation provides:

- **GCounter / PNCounter**: Monotonic and bidirectional counters
- **LWW-Register**: Last-writer-wins for scalar values
- **OR-Set**: Observed-remove set with add-wins semantics
- **MV-Register**: Multi-value register preserving concurrent writes
- **Document**: Composable CRDT document with nested fields

CRDT operations carry *ciphertext*, not plaintext. Sync peers relay opaque encrypted bytes without decrypting. Only the user's device (which holds the DEK after biometric/passkey unlock) can read the actual values.

2.4 Chain Anchoring

The chain stores only:

- DID / identity roots
- Key handles and rotation events
- Capability / policy state
- Sync checkpoint Merkle roots
- Audit receipts
- Payment / metering records

A Merkle root is computed over the CRDT operation log:

$$\text{root} = \text{SHA-256}\left(\text{orgID} \| \text{userID} \| \bigoplus_{i=1}^{n} (\text{key}_i \| \text{ct}_i)\right) \tag{3}$$

where ct_i is the ciphertext of the i-th entry. This root is committed to the I-Chain (IdentityVM) as an anchor receipt, providing tamper-evidence without revealing data.

2.5 On-Chain VMs

The trust plane runs on three Lux Network VMs deployed as a private subnet:

- **I-Chain (IdentityVM)**: DIDs, verifiable credentials, OIDC sessions, anchor receipts. Supports `did:lux`, `did:web`, `did:key` methods.
- **K-Chain (KeyVM)**: ML-KEM post-quantum key management. Stores key handles (never plaintext keys). Enforces rotation policy, access control, and audit logging.
- **T-Chain (ThresholdVM)**: CGGMP21 (ECDSA), FROST (EdDSA), BLS threshold signing. JWTs are threshold-signed—no single signing key exists.

3 SDK Design

The SDK exposes five primitives:

1. **Identity**: `OpenUser(userID)` — resolve DID, unwrap DEK, bind device.
2. **Key Access**: `GetShardKey()` — unwrap via HSM or K-Chain threshold.
3. **Local DB**: `Put(key, value)`, `Get(key)`, `Delete(key)` — encrypted SQLite.
4. **Sync**: `Sync()`, `Merge(ops)` — CRDT over ZAP (8.7µs RTT).
5. **Anchor**: `Anchor()` — Merkle root to chain, returns audit receipt.

Usage is minimal:

```
v, _ := vault.Open(vault.SDKConfig{
    DataDir:   "./data",
    MasterKEK: masterKey,
    OrgID:     "my-org",
})
session, _ := v.OpenUser("alice")
session.Put("prefs", []byte(`{"theme":"
    dark"}`))
session.Sync()
receipt, _ := session.Anchor()
```

4 The Firebase Replacement

Web5 replaces centralized Backend-as-a-Service:

Firebase	Web5
Auth	DID + capability/session gateway
Firestore	Encrypted SQLite shard
Offline sync	CRDT
Storage	Content-addressed blob store
Functions	Workers on CRDT ops / chain receipts

The migration path is: Firebase app → local-first app with chain-anchored trust. Not: Firebase app → smart contracts everywhere.

5 Security Analysis

5.1 Key Compromise

If a user's DEK is compromised, only that user's shard is exposed. The org KEK and other users' DEKs remain safe (HMAC-SHA256 is a PRF). Device revocation zeros the DEK immediately.

5.2 Sync Privacy

CRDT operations carry ciphertext. Relay nodes see encrypted bytes and metadata (key names, timestamps) but not values. For metadata privacy, key names can also be encrypted or hashed.

5.3 Chain Minimality

The chain never stores row data, keys, or plaintext. Merkle roots are commitments—they prove state integrity without revealing state content.

5.4 Post-Quantum Readiness

ML-KEM-768 wraps the Master KEK for distribution. Symmetric encryption (AES-256-GCM) is quantum-resistant by design. Threshold signing uses FROST (Ed25519) and CGGMP21 (secp256k1) today, with Corona (lattice-based) as the post-quantum threshold upgrade path.

6 Implementation Status

The system is implemented across multiple open-source repositories:

- `hanzoai/base` — Hanzo Base framework with vault plugin (18 tests passing)
- `luxfi/node` — 14 VMs including IdentityVM, KeyVM, ThresholdVM (all building, tested)
- `luxfi/hsm` — Cloud HSM integration (AWS, GCP, Azure)
- `luxfi/mpc` — Threshold signing (CGGMP21, FROST, BLS)
- `luxfi/zap` — Zero-copy transport (8.7µs latency)
- `zooai/universe` — Reference deployment with auth subnet

All code is open source under permissive licenses.

7 Conclusion

Web5 is not "put apps on-chain." Web5 is "put trust on-chain, keep state local, sync privately, and make identity portable."

Every web2 application should become local-first, encrypted by default, user-portable, offline-capable, chain-auditable, permissionlessly hosted, and cryptographically interoperable. The architecture described here—per-user SQLite shards, CRDT sync, ML-KEM key wrapping, and chain anchoring—achieves this without sacrificing the performance, privacy, or usability that users expect.

The chain is the root of trust, settlement, and interoperability. Not the database.

References

[1] R. Hipp, "SQLite: Past, Present, and Future," *Proc. VLDB Endow.*, vol. 15, no. 12, 2022.

[2] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, Aug. 2024.

[3] M. Shapiro et al., "Conflict-Free Replicated Data Types," *SSS 2011*, LNCS 6976, pp. 386–400.

[4] M. Kleppmann et al., "Local-First Software: You Own Your Data, in spite of the Cloud," *Onward! 2019*.